

Authorization 授权.

- Make sure that the users see the data that they are supposed to see
- Guard the database against modifications by malicious users
- Users have **privilege**; user can only operate on data they are authorized

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> i_id

特权.

Privileges in SQL

- **select:** allows read access to relation, or the ability to query using the view
 - Example: grant users U_1 , U_2 , and U_3 **select** authorization on the *instructor* relation:

$\text{grant } \text{select} \text{ on } \text{instructor} \text{ to } U_1, U_2, U_3$

privileges relation user.
- **insert:** the ability to insert tuples
- **update:** the ability to update using the SQL update statement
- **delete:** the ability to delete tuples.
- **all privileges:** used as a short form for all the allowable privileges

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Authorization

- **update student S**
set tot_cred = (select sum(credits)
from takes, course
where takes.course_id = course.course_id and
S.ID= takes.ID.and
takes.grade <> 'F' and
takes.grade is not null);

select:

update:

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> year building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> i_id

Authorization to select a set of tuples/attributes

- Authorization to a user to instructors from Geology

view

- **create view** *geo_instructor* **as**
(**select** *
from *instructor*
where *dept_name* = 'Geology');

- **grant select on** *geo_instructor* **to** *geo_staff*
- Suppose that a *geo_staff* member issues
 - **select ***
from *geo_instructor*;
- What if
 - *geo_staff* does not have permissions on *instructor*?
 - Creator of view did not have some permissions on *instructor*?

not necessary update privilege.

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> i_id

Authorization Specification in SQL

- The **grant** statement is used to confer authorization
grant <privilege list> **on** <relation or view > **to** <user list>

- <user list> is:

- a user-id
- public**, which allows all valid users the privilege granted

- A role 如: 管理员, 经理
人物, 不是特定人

- Example:

- grant select on department to** Amit, Satoshi

@ operator must have
privilege

@ relation
can be - private

to---

- Granting a privilege on a view does not imply granting any privileges on the underlying relations.

- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).

		teaches		section	takes		student	course	advisor	
department	instructor	ID	course_id	course_id	ID	course_id	ID	course_id	s_id	i_id
dept_name	name	sec_id	sec_id	sec_id	sec_id	sec_id	name	title		
building	dept_name	semester	semester	semester	semester	semester	dept_name	dept_name		
budget	salary	year	year	year	year	year	tot_cred	credits		
				building						
				room_number						
				time_slot_id						
					grade					

Roles Example

- **create role** instructor;
- **grant** *instructor* **to** Amit;
- Privileges can be granted to roles:
 - **grant select on takes to instructor**;
- Roles can be granted to users, as well as to other roles
 - **create role** *teaching_assistant*
 - **grant** *teaching_assistant* **to** *instructor*;
 - *Instructor* inherits all privileges of *teaching_assistant*
- **Chain** of roles
 - **create role** *dean*;
 - **grant** *instructor* **to** *dean*;
 - **grant** *dean* **to** Satoshi;

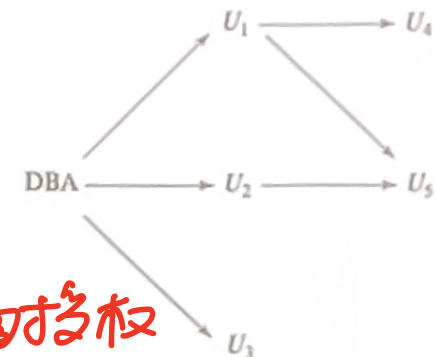
department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Authorization

- Relation creator is the owner
- Owner has all privileges and may grant privileges

Example: grant select on department to Amit, Satoshi

- Grant the same or lesser privileges to other users



撤销。

→ 会链式影响后面授权

- Revoke privileges
 - revoke select on department from Amit, Satoshi cascade;
 - revoke select on department from Amit, Satoshi restrict;

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Wrap up

- **Basic Queries:** SELECT A1, A2, ..., An FROM r1, r2, ..., rm WHERE P
- **String Operations:** LIKE '%pattern%'. Wildcards: _ (single character), % (multiple characters)
- **Aggregation**
 - GROUP BY A1, A2, ..., An
 - Filtering groups: HAVING P
- **Nested Queries**
 - Used in SELECT, FROM, WHERE clauses
- **Data Modification**
 - INSERT, DELETE, UPDATE
- **Joins**
 - NATURAL JOIN
 - INNER JOIN
 - (LEFT, RIGHT, FULL) OUTER JOIN
 - JOIN ... ON condition
 - JOIN ... USING (column)
- **Data Types:** Ensuring correct storage formats
- **Views:** Virtual tables for abstraction
- **Integrity Constraints:** Enforcing data consistency
- **Authorization:** Managing user access and permissions
 - GRANT (Assign privileges)
 - REVOKE (Remove privileges)

Accessing SQL from a Programming Language

A database programmer must have access to a general-purpose programming language for at least two reasons

- Not all queries can be expressed in SQL, since SQL does not provide the full expressive power of a general-purpose language.
- Non-declarative actions -- such as printing a report, interacting with a user, or sending the results of a query to a graphical user interface -- cannot be done from within SQL.

JDBC

- **JDBC** is a Java API for communicating with database systems supporting SQL.
- JDBC supports a variety of features for querying and updating data, and for retrieving query results. 元数据.
- JDBC also supports metadata retrieval.
- Model for communicating with the database:
 - Open a connection
 - Create a “statement” object
 - Execute queries using the statement object to send queries and fetch results
 - Exception mechanism to handle errors

JDBC Code

```
public static void JDBCexample(String dbid, String userid, String passwd)
{
    try (Connection conn = DriverManager.getConnection(
        "jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
        Statement stmt = conn.createStatement();
    )
    {
        ... Do Actual Work ....
    }
    catch (SQLException sqle) {
        System.out.println("SQLException : " + sqle);
    }
}
```

NOTE: Above syntax works with Java 7, and JDBC 4 onwards.

Resources opened in “try (....)” syntax (“try with resources”) are automatically closed at the end of the try block

JDBC Code (Cont.)

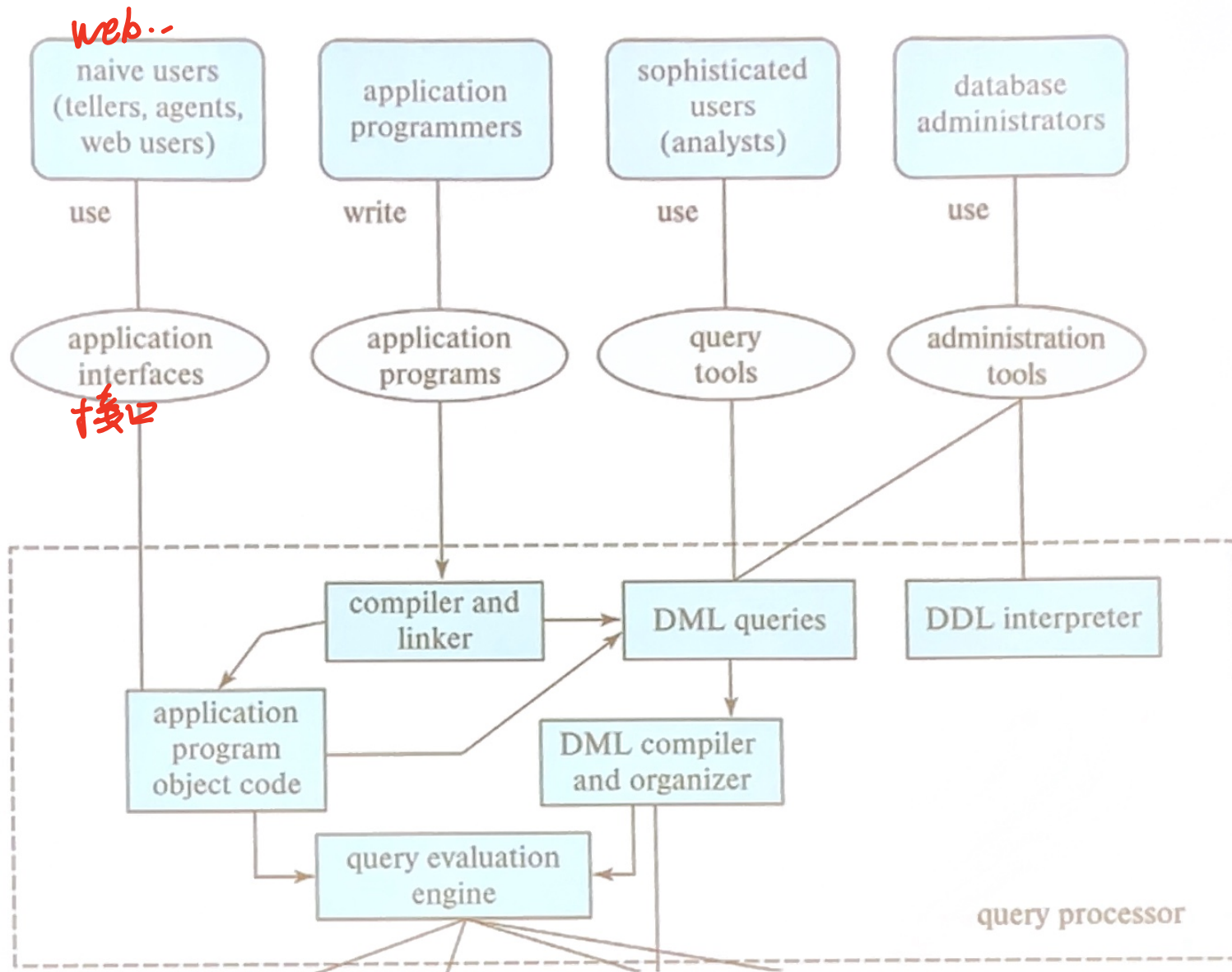
- Update to database

```
try {  
    stmt.executeUpdate(  
        "insert into instructor values('77987', 'Kim', 'Physics', 98000)");  
} catch (SQLException sqle)  
{  
    System.out.println("Could not insert tuple. " + sqle);  
}
```

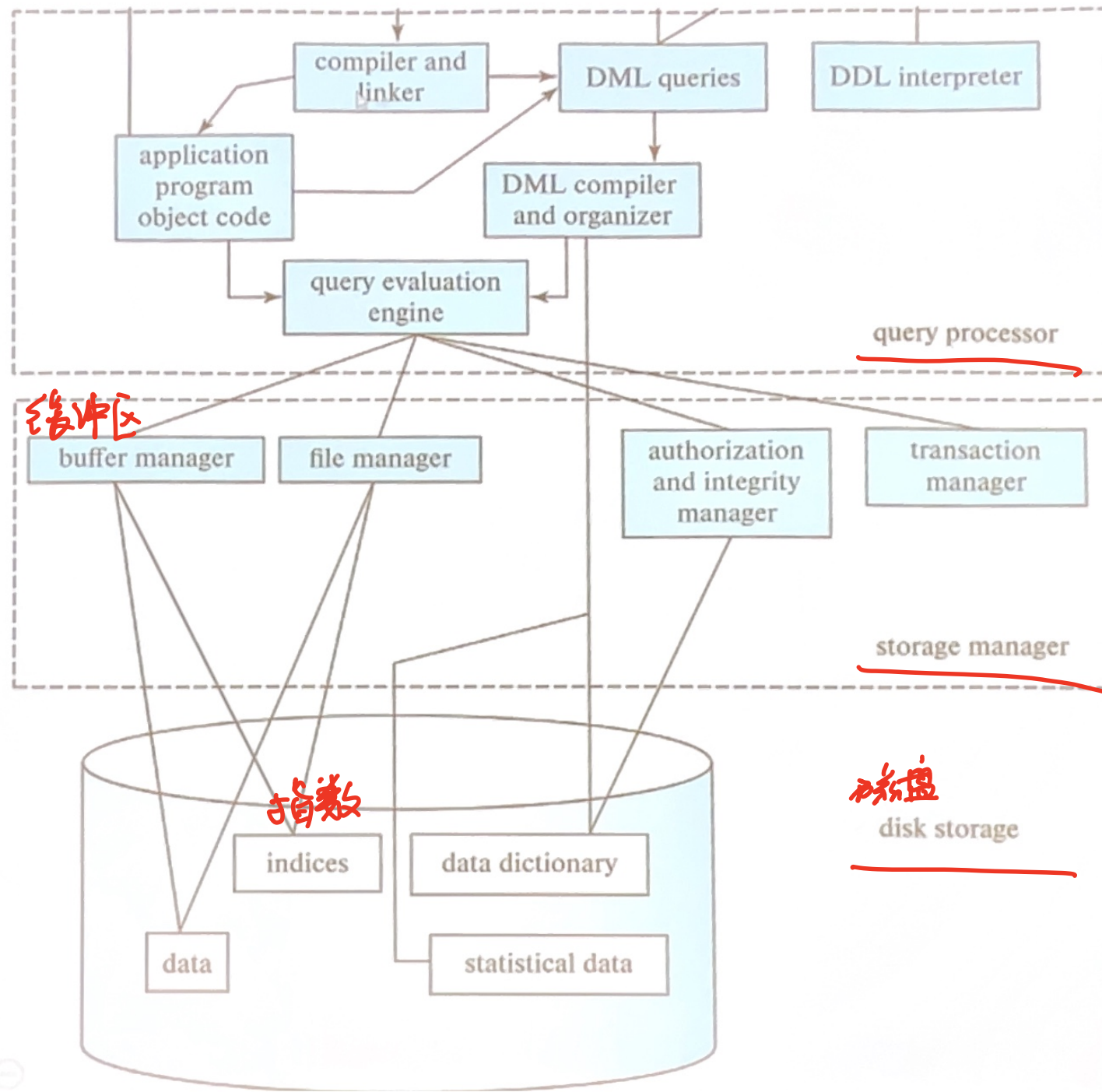
- Execute query and fetch and print results

```
ResultSet rset = stmt.executeQuery(  
    "select dept_name, avg (salary)  
    from instructor  
    group by dept_name");  
while (rset.next()) {  
    System.out.println(rset.getString("dept_name") + " " + rset.getFloat(2));  
}
```

Database Users



Database Architecture (Centralized/Shared-Memory)



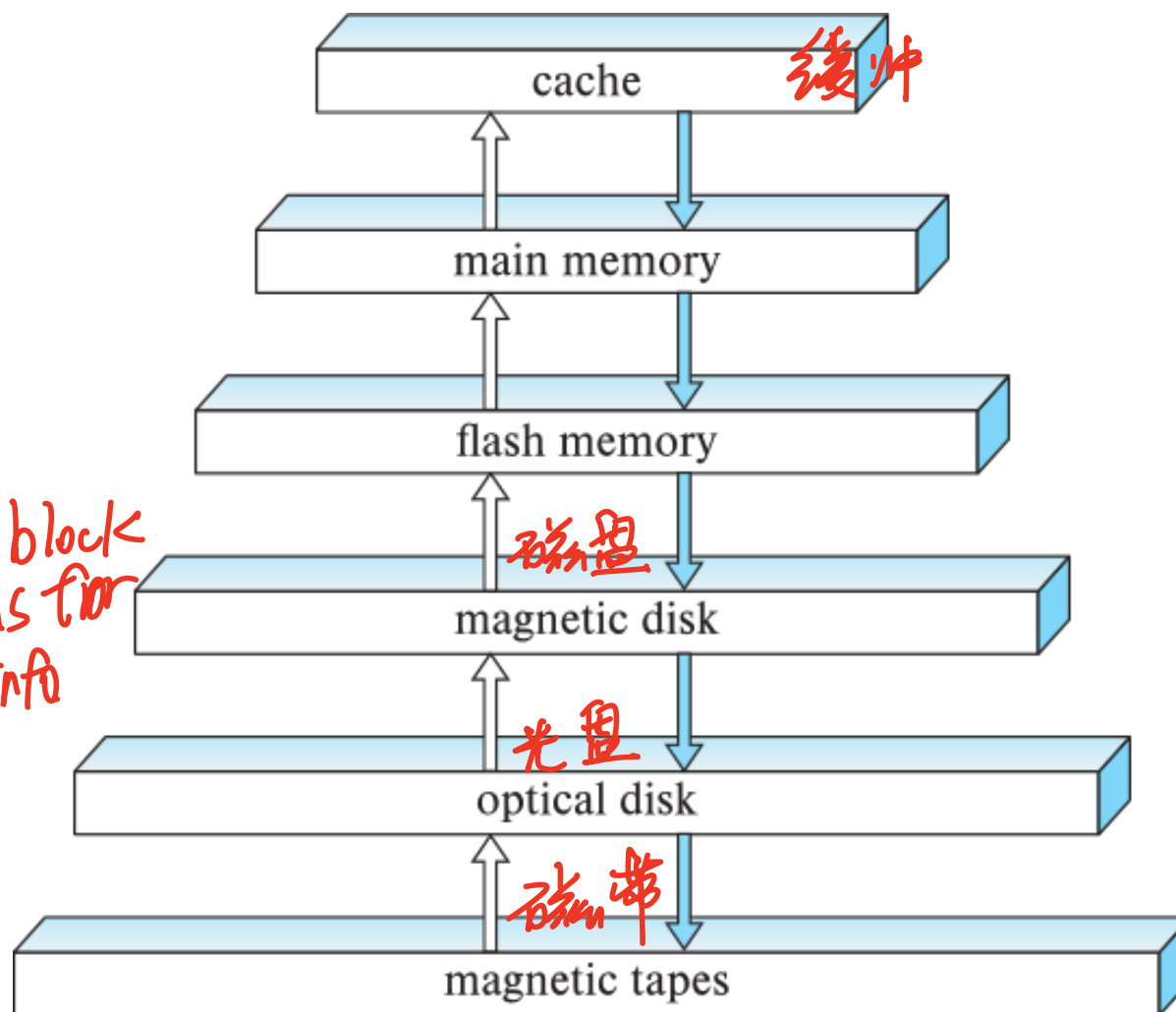
Storage

Miao Qiao

The University of Auckland



Storage Hierarchy



缓冲

磁盘

光盘

磁带

expensive.

fast

↑

↓

cheap

slow

byte

in block

I/O model

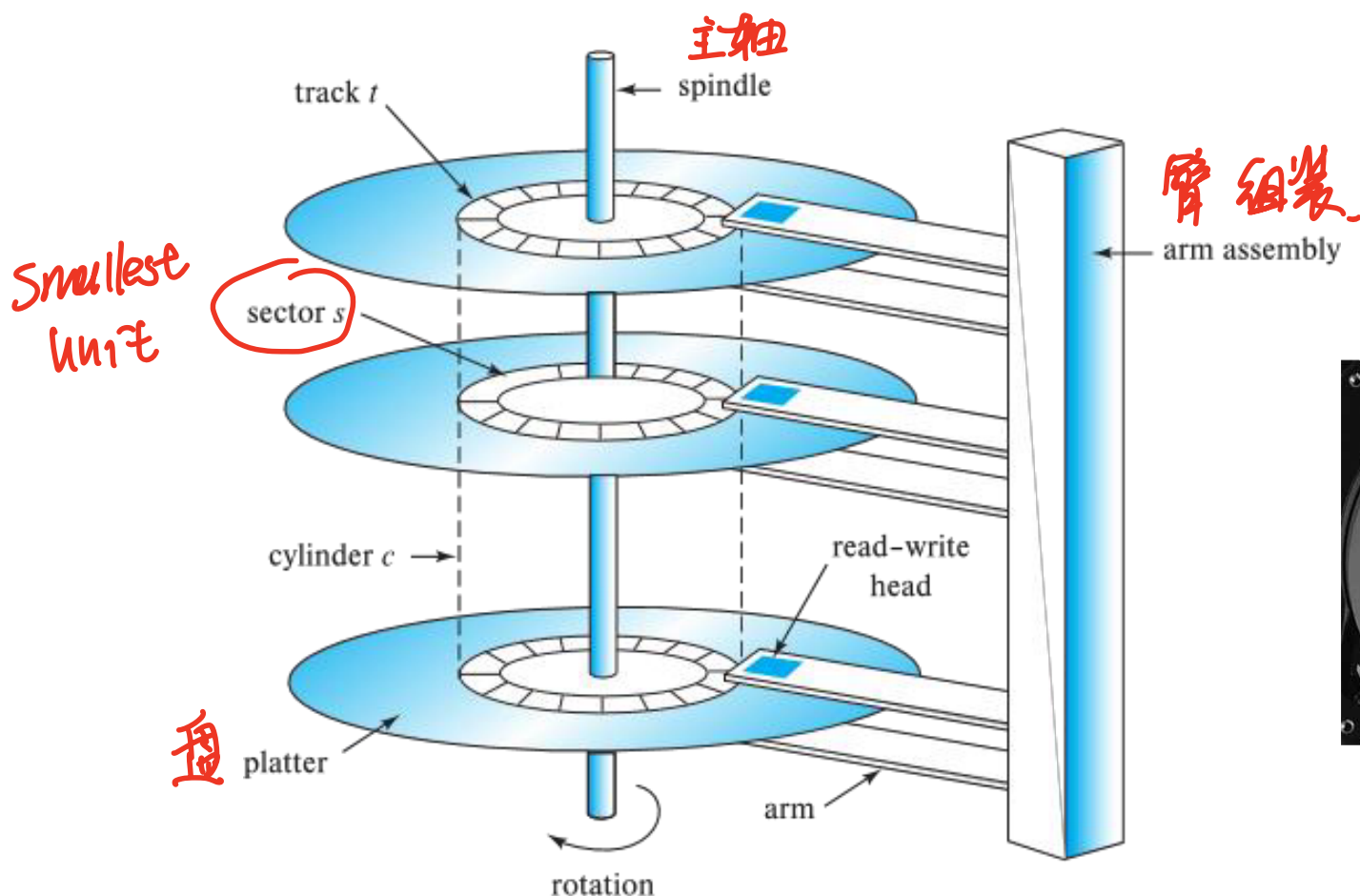
↑ 对比

RAM model

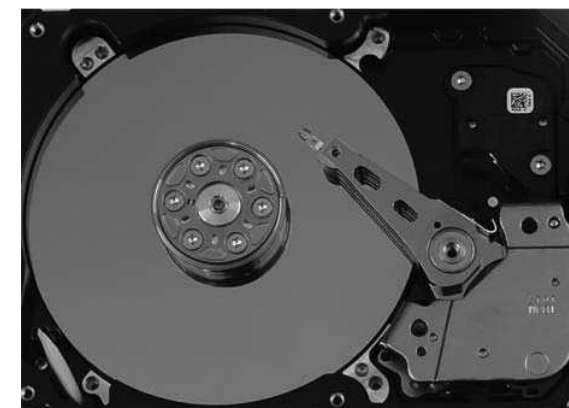
use block transfer info

all store in main memory.
Google Data Center

Magnetic Hard Disk Mechanism



顺序读取比
随机读取快得多
(seek)



Schematic diagram of magnetic disk drive

Photo of magnetic disk drive

Performance Measures

- **Disk block**
- **Sequential access pattern**
 - Magnetic disks: 25 to 200 MB per second max rate on
 - SSD: 400 MB/sec for SATA3, 2 to 3 GB/sec using NVMe PCIe *SSD → limit number over write*
- **Random access pattern**
- **I/O operations per second (IOPS)**
 - Number of random block reads that a disk can support per second
 - Magnetic disks (4K block): 50 to 200 IOPS
 - SSD (4K block):
 - 10,000 reads per second (10,000 IOPS)
 - 40,000 writes per second
- **Mean time to failure (MTTF)** – the average time the disk is expected to run continuously without any failure. *不出现故障的平均时间*
 - Typically 3 to 5 years
- **Hybrid disks**
 - combine small amount of flash cache with larger magnetic disk

RAID

冗余 阵列

■ RAID: Redundant Arrays of Independent Disks

- manage a large numbers of disks, providing a view of a single disk of

- high capacity, } 多磁盘
- high speed,
- high reliability → 冗余, duplicate data.

- $\text{Prob}(\text{a disk out of a set of } N \text{ disks fails}) \gg \text{Prob}(\text{a single disk fails})$.

- E.g., a system with 100 disks, each with MTTF of 100,000 hours (approx. 11 years), will have a system MTTF of 1000 hours (approx. 41 days)

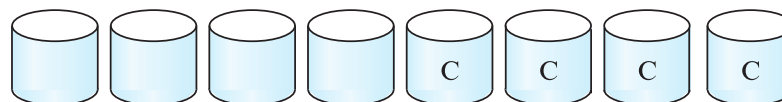
Improvement of Reliability via Redundancy

重建失去信息

- ① type
 - **Redundancy** – store extra information that can be used to rebuild information lost in a disk failure
 - E.g., **Mirroring** (or **shadowing**)
 - **Mean time to data loss** depends on mean time to failure, and **mean time to repair**
 - E.g., MTTF of 100,000 hours, mean time to repair of 10 hours gives mean time to data loss of 500×10^6 hours (or 57,000 years) for a mirrored pair of disks (ignoring dependent failure modes)



(a) RAID 0: nonredundant striping



(b) RAID 1: mirrored disks

1 byte = 8 bits

②

Improvement in Performance via Parallelism

- Two main goals of parallelism in a disk system:
 1. Load balance multiple small accesses to increase **throughput**
 2. Parallelize large accesses to reduce **response time**.

- Improve transfer rate by striping data across multiple disks.

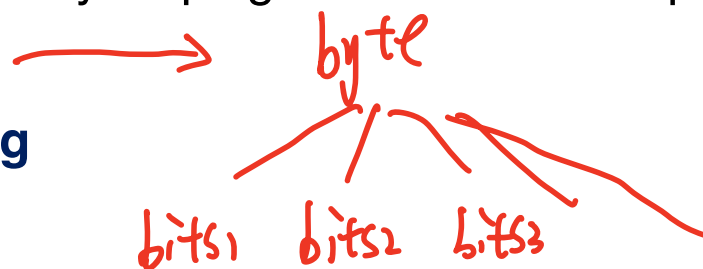
- Bit-level striping**

- Block-level striping**

- Parity blocks**

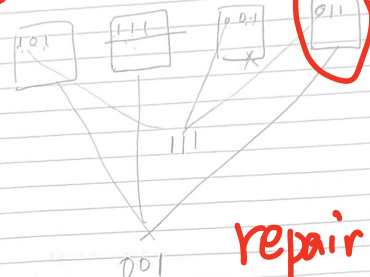
bits
↓
block

无数据 奇偶校验块



↓ read. five together fast

bits1 bits2 ...



(c) RAID 5: block-interleaved distributed parity

repair by

other 3

only can repair 1.

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

写的有瓶颈，两边一起update不行

Choice of RAID Level

- Factors in choosing RAID level
 - Monetary cost
 - Performance: Number of I/O operations per second, and bandwidth during normal operation
 - Performance during failure
 - Performance during rebuild of failed disk
 - Including time taken to rebuild failed disk
- Level 0 is used only when data safety is not important
 - E.g., data can be recovered quickly from other sources
- Level 1 provides much better write performance than level 5
 - Level 5 requires at least 2 block reads and 2 block writes to write a single block, whereas Level 1 only requires 2 block writes

File Organization

- The database is stored as a collection of *files*.
- Each file is a sequence of *records*.
- A record is a sequence of fields.
- One approach
 - Assume record size is fixed
 - Each file has records of one particular type only
 - Different files are used for different relations

This case is easiest to implement; will consider variable length records later

- We assume that records are smaller than a disk block.

Fixed-Length Records

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Deletion of record 3:

1. move records $3 + 1, \dots, n$ to $3, \dots, n - 1$, or
2. move record n to 3, or
3. do not move records, but link all free records on a *free list*

Fixed-Length Records

- move records $3 + 1, \dots, n$ to $3, \dots, n - 1$

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

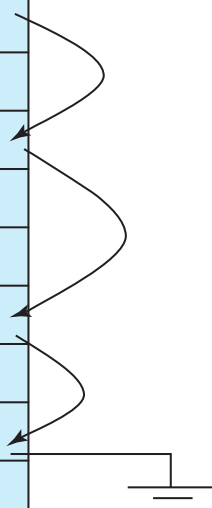
- move record n to i

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Fixed-Length Records

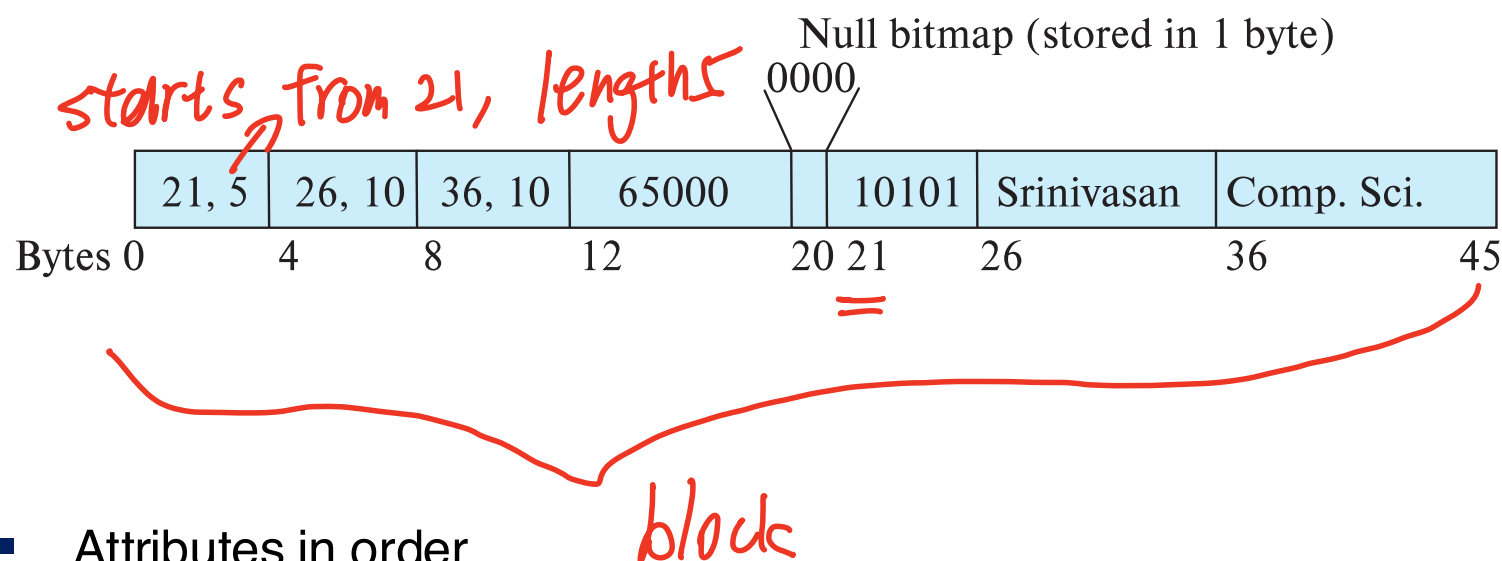
- do not move records, but link all free records on a *free list*

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



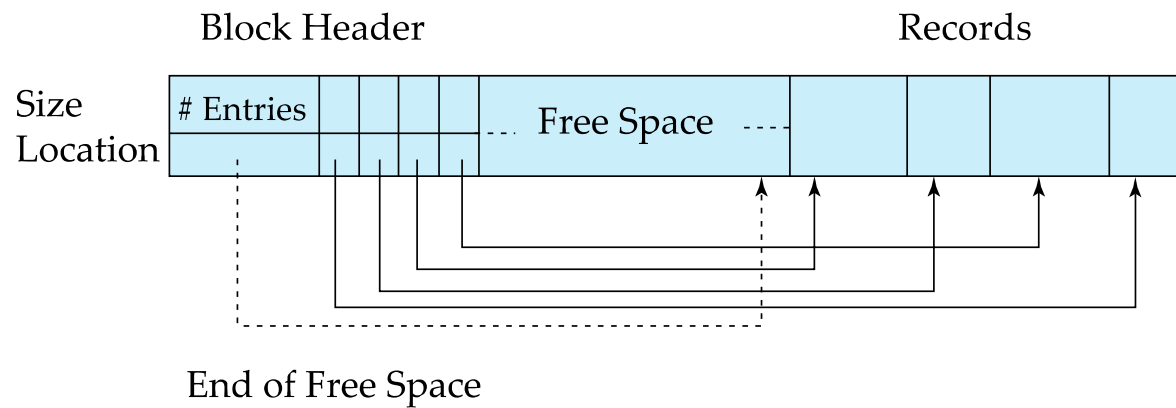
Variable-Length Records

- Variable lengths such as **varchar**



- Attributes in order
- (offset, length): actual data stored after all fixed-length attributes

Variable-Length Records: Slotted Page Structure



Organization of Records in Files

- **Heap** – record can be placed anywhere in the file where there is space
- **Sequential** – store records in sequential order, based on the value of the search key of each record
- **B⁺-tree file organization**
 - Ordered storage even with inserts/deletes
 - More on this in Indexing
- **Hashing** – a hash function computed on search key; the result specifies in which block of the file the record should be placed
 - More on this in Indexing



Sequential File Organization

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	
-------	-------	-------	-------	--

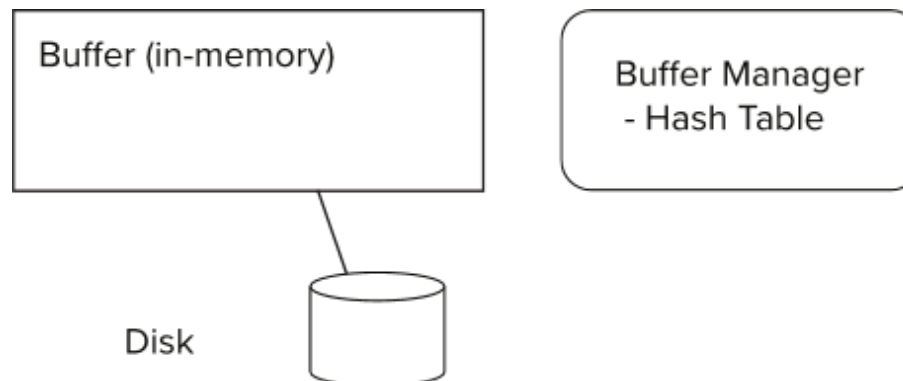
Data Dictionary Storage

The **Data dictionary** (also called **system catalog**) stores **metadata** -- data about data, such as

- Relations
 - names
 - names, types and lengths of attributes
 - names and definitions of views
 - integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data
 - number of tuples in each relation
- Physical file organization
 - How relation is stored (sequential/hash/...)
 - Physical location of relation
- Information about indices...

Storage Access

- Blocks are units of both storage allocation and data transfer.
- **I/O model** - database system seeks to minimize the number of block transfers between the disk and memory.
- We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.
- **Buffer** – portion of main memory available to store copies of disk blocks.
- **Buffer manager** – subsystem responsible for allocating buffer space in main memory.



Buffer Manager

- Buffer replacement strategy (details coming up!) 缓冲区替换策略.
- Pinned block 图块
 - Pin
 - Unpin
 - Multiple concurrent pin/unpin operations possible
 - Keep a pin count, buffer block can be evicted only if pin count = 0
- Shared and exclusive locks on buffer
 - Locking rules:
 - Only one process can get exclusive lock at a time
 - Shared lock cannot be concurrently with exclusive lock
 - Multiple processes may be given shared lock concurrently

Buffer-Replacement Policies

- **Least recently used (LRU strategy)**
 - Idea behind LRU – use past pattern of block references as a predictor of future references
 - LRU can be bad for some queries
 - Example of bad access pattern for LRU: when computing the join of 2 relations r and s by a nested loops

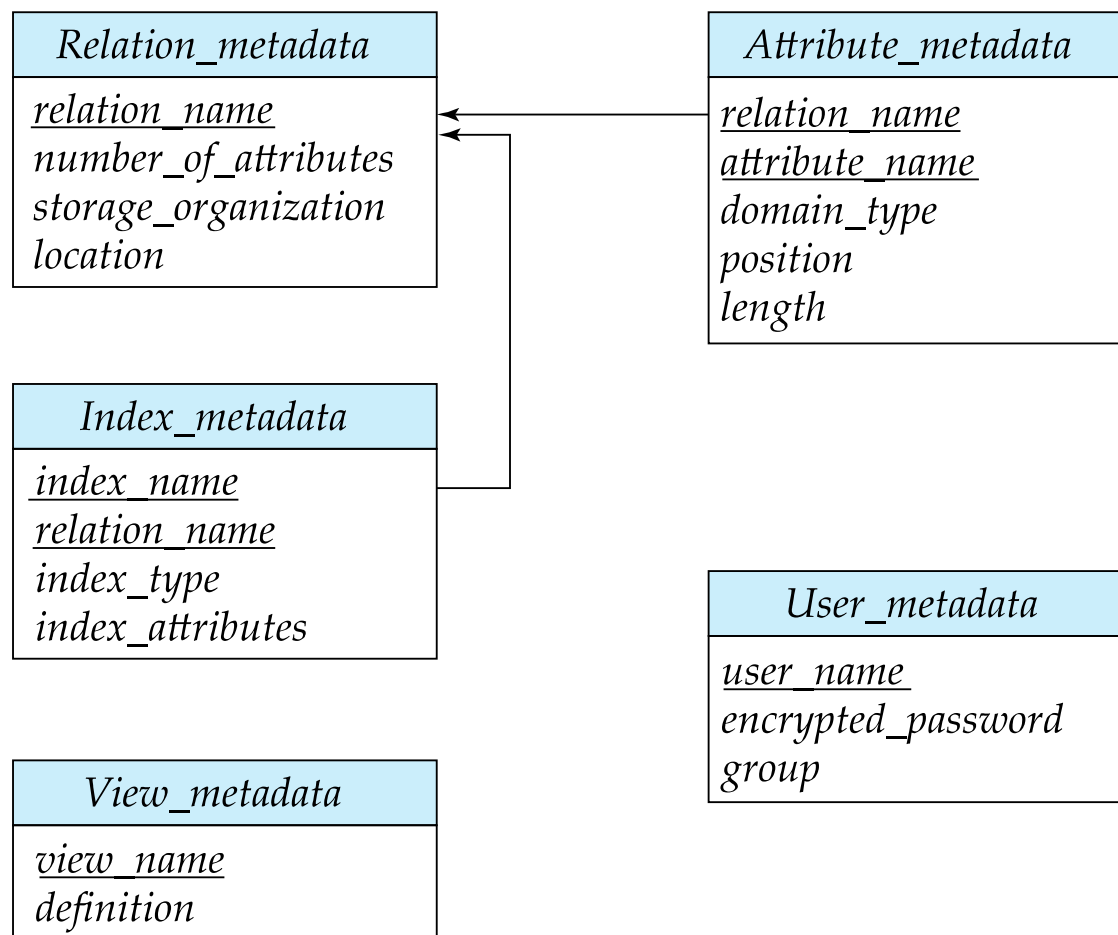
```
for each tuple  $tr$  of  $r$  do
  for each tuple  $ts$  of  $s$  do
    if the tuples  $tr$  and  $ts$  match ...
```
- **Most recently used (MRU) strategy**
 - System must pin the block currently being processed.
 - After the final tuple of that block has been processed, the block is unpinned, and it becomes the most recently used block.
- **Buffer manager** can use statistical information regarding the probability that a request will reference a particular relation

FIN

Any questions?

Relational Representation of System Metadata

- Relational representation on disk
- Specialized data structures designed for efficient access, in memory



Optimization of Disk Block Access (Cont.)

- Buffer managers support **forced output** of blocks for the purpose of recovery (more in Chapter 19)
- **Nonvolatile write buffers** speed up disk writes by writing blocks to a non-volatile RAM or flash buffer immediately
 - *Writes can be reordered to minimize disk arm movement*
- **Log disk** – a disk devoted to writing a sequential log of block updates
 - Used exactly like nonvolatile RAM
 - Write to log disk is very fast since no seeks are required
- **Journaling file systems** write data in-order to NV-RAM or log disk
 - Reordering without journaling: risk of corruption of file system data

Column-Oriented Storage

- Also known as **columnar representation**

Stor

Exa

10101
12121
15151
22222
32343
33456
45565
58583
76543
76766
83821
98345

Srinivasan
Wu
Mozart
Einstein
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim

Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

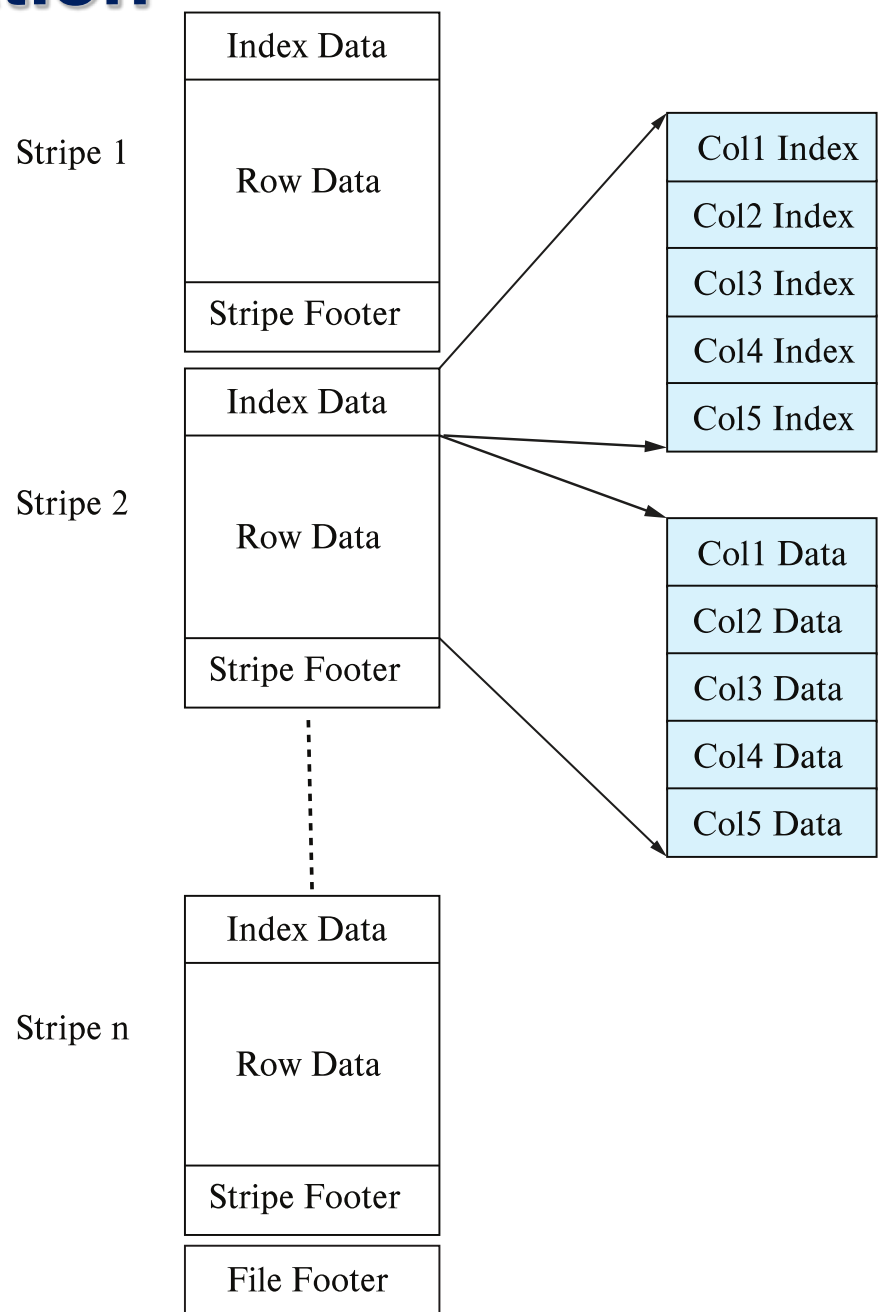
65000
90000
40000
95000
60000
87000
75000
62000
80000
72000
92000
80000

Columnar Representation

- Benefits:
 - Reduced IO if only some attributes are accessed
 - Improved CPU cache performance
 - Improved compression
 - **Vector processing** on modern CPU architectures
- Drawbacks
 - Cost of tuple reconstruction from columnar representation
 - Cost of tuple deletion and update
 - Cost of decompression
- Columnar representation found to be more efficient for decision support than row-oriented representation
- Traditional row-oriented representation preferable for transaction processing
- Some databases support both representations
 - Called **hybrid row/column stores**

Columnar File Representation

- ORC and Parquet: file formats with columnar storage inside file
- Very popular for big-data applications
- Orc file format shown on right:



Storage Organization in Main-Memory Databases

- Can store records directly in memory without a buffer manager
- Column-oriented storage can be used in-memory for decision support applications
 - Compression reduces memory requirement

